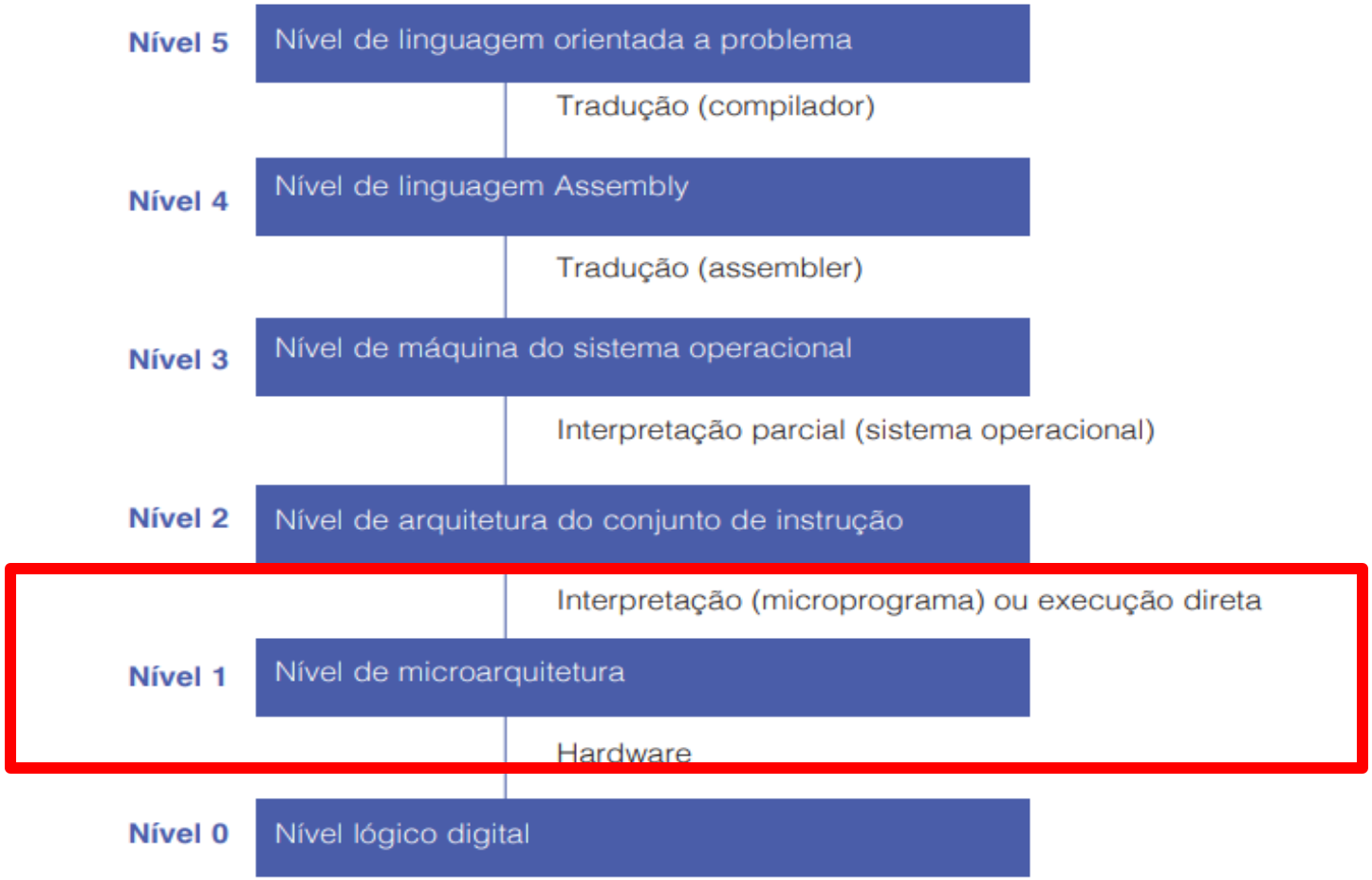
Nível de microarquitetura

Controle de fluxo
Sequência de instruções.

Figura 1.2 Um computador com seis níveis. O método de suporte para cada nível é indicado abaixo dele (junto com o nome do programa que o suporta).



O nível de microarquitetura

- É o nível acima do **lógico digital** chamado de nível de **microarquitetura**.
- Sua função é executar o nível ISA (*Instruction Set Architecture* – **arquitetura do conjunto de instruções**) acima dele.

O nível de microarquitetura

- O projeto do nível de microarquitetura **depende da ISA que está sendo implementada**, bem como das metas de custo e desempenho do computador.
- *No nível de microarquitetura,* podemos ter uma coleção de 8 a 32 registradores que formam uma memória local e um circuito chamado ULA – Unidade Lógica e Artitmética, que é capaz de realizar operações aritméticas simples.

O nível de microarquitetura

- Muitas ISAs modernas, em particular projetos RISC, **têm instruções simples** que podem ser executadas em um **único ciclo de *clock***.
  - ISAs mais complexas, como a Core i7, podem exigir muitos ciclos para executar uma única instrução.
- Executar uma instrução pode requerer **localizar os operandos na memória**, **ler esses operandos** e **armazenar resultados de volta na memória**.
  - A sequência de operações dentro de uma única instrução muitas vezes leva a uma abordagem de controle diferente da adotada para ISAs simples.

O nível de microarquitetura

- Para explicar, temos um exemplo:
  - Para nossa ISA, escolhemos um subconjunto da Java Virtual Machine.
  - Esse subconjunto contém somente instruções com **inteiros**
  - Vamos denominar de **IJVM** para enfatizar que ele trata somente de **inteiros.**
- A arquitetura tem algumas instruções complexas e outras simples.
- Embora a IJVM seja pequena, é um bom ponto de partida para descrever o **controle de fluxo e a sequência de instruções.**

O nível de microarquitetura

- A microarquitetura contem um microprograma (em ROM) cuja tarefa é **buscar, decodificar e executar instruções** IJVM.

O nível de microarquitetura

- O microprograma tem um conjunto de variáveis denominado **estado do computador,** que pode ser acessado por todas as funções.
- Cada função altera ao menos algumas das variáveis que compõem o estado.
  - Por exemplo, o registrador **contador de programa** (Program Counter – PC) é parte do estado.
- PC indica a localização da memória que contém a **próxima** função (instrução ISA) a ser executada.
  - Durante a execução de cada instrução, o PC é incrementado para indicar a próxima instrução a ser executada.

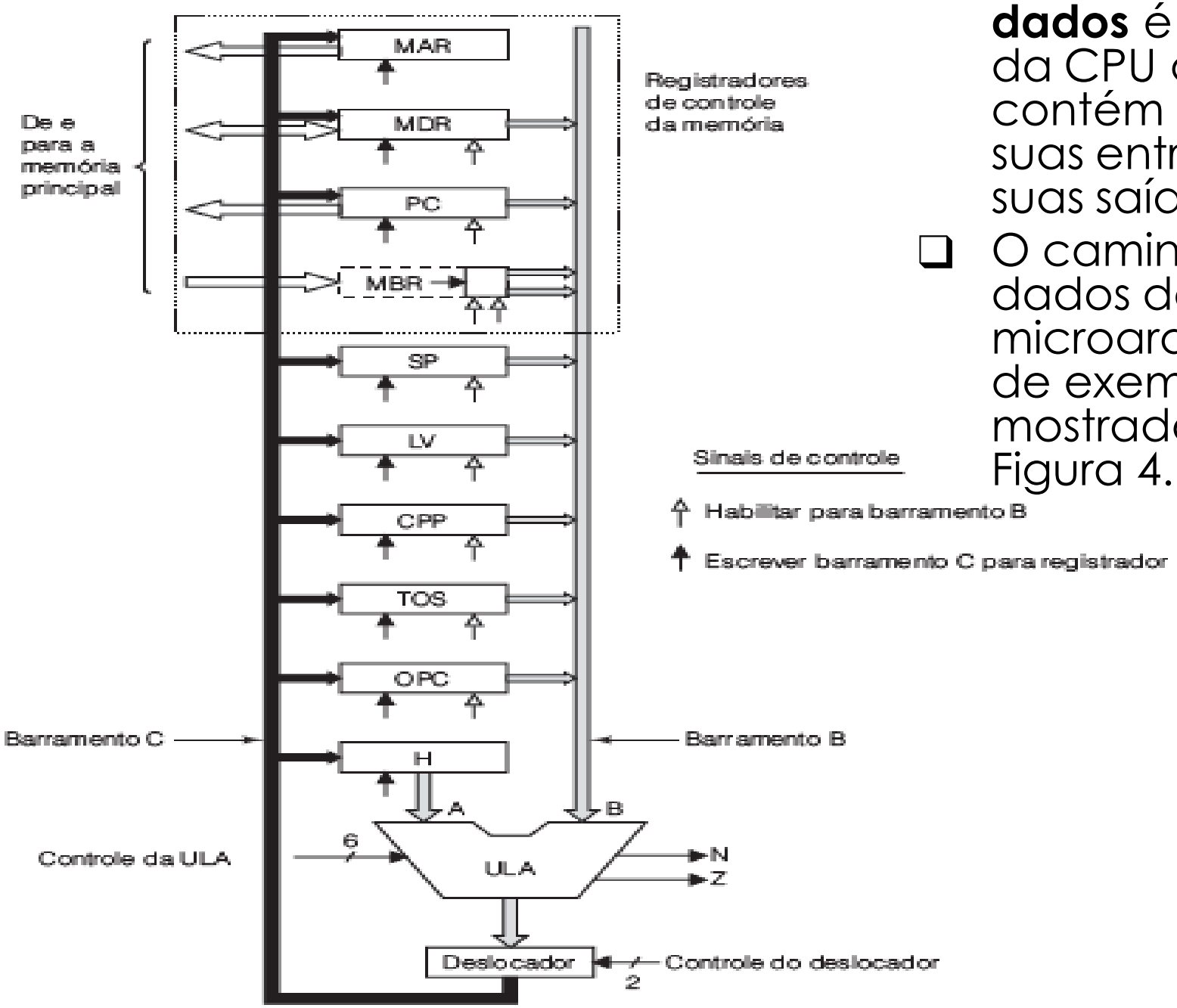
O nível de microarquitetura

- Instruções IJVM são curtas e fáceis.
- Cada instrução tem um ou dois campos, e cada um tem alguma finalidade específica.
  - O primeiro campo de toda instrução é o ***opcode*** (código de operação), que **identifica a instrução**, informando a operação, ex: ***ADD*** ou outra.
  - O 2º campo **especifica o operando.** Por exemplo, instruções que acessam uma variável local precisam de um campo que identifique **qual variável.**
- Esse modelo de execução é o **ciclo buscar-decodificar-executar.**

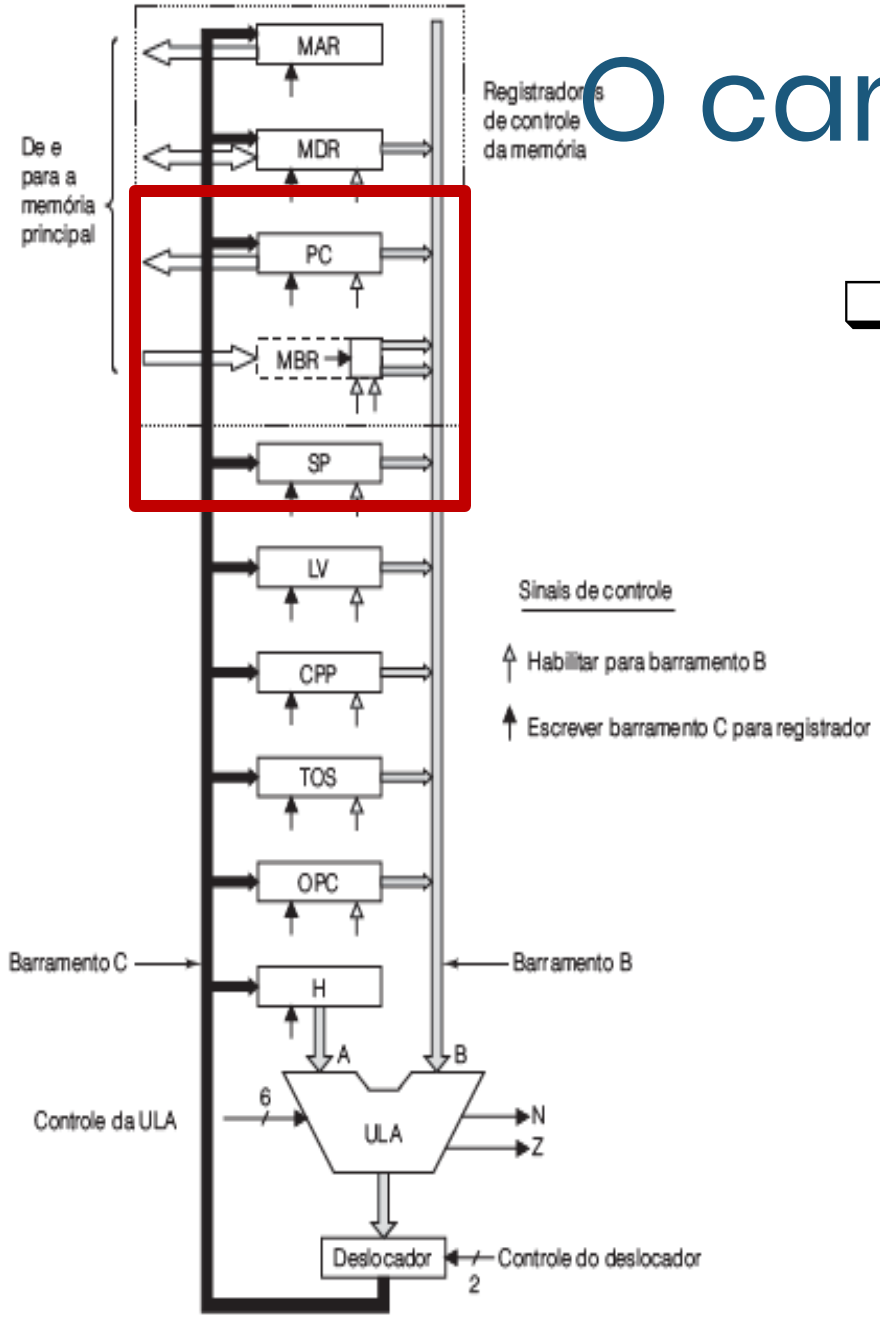
Registradores

- MAR (Memory Address Register): registrador de endereçamento de memória (32 bits).
- MDR (Memory Data Register): registrador de dados de memória, leitura ou escrita na memória (apontado por MAR).
- PC (Program Counter): indica a posição de memória (endereço) que contém a próxima instrução a ser executada.
- MBR (Memory Byte Register) : Porta de dados da memória (8bits), apenas leitura da memória
- SP (Stack Pointer): Aponta para o topo da pilha
- LV (Local Variable): Aponta para a base das variáveis locais dentro da pilha
- CPP (Constant Pool Pointer): Aponta para a base da área de constantes.
- TOS e OPC: Registradores Temporários.
- H: acumulador

- O **caminho de dados** é a parte da CPU que contém a ULA, suas entradas e suas saídas.
- O caminho de dados de nossa microarquitetura de exemplo é mostrado na Figura 4.1.

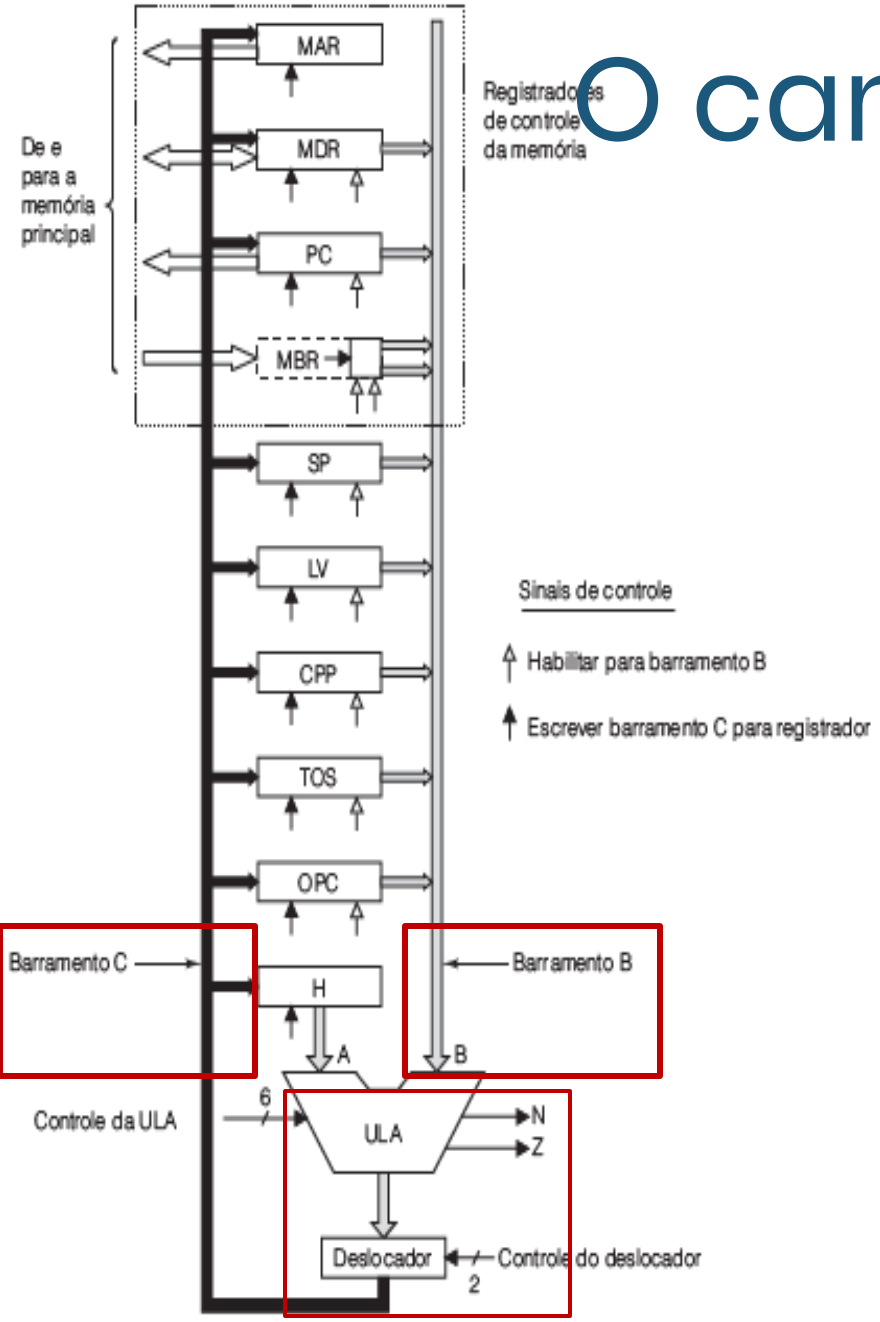


O caminho de dados



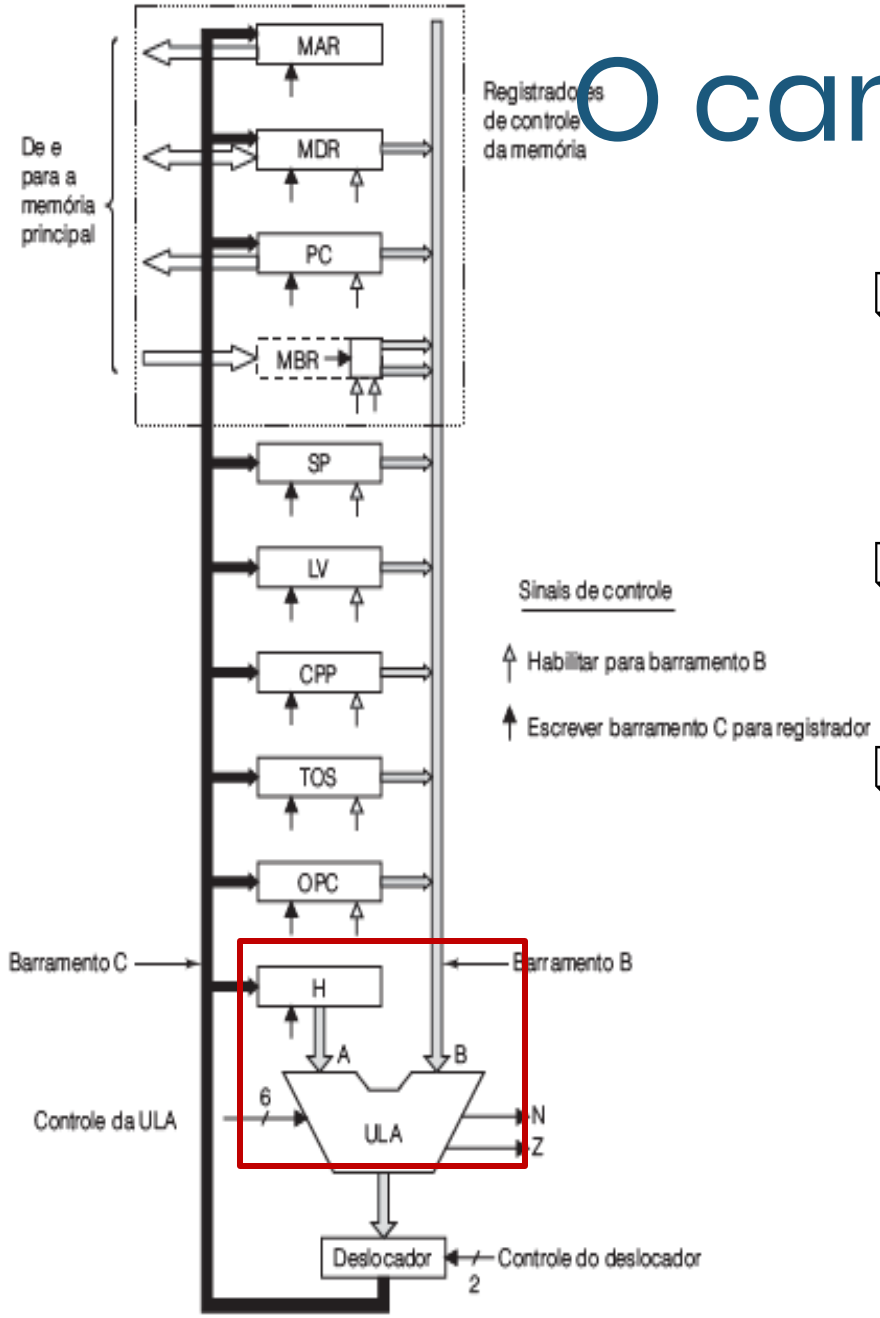
- O caminho de dados contém **registradores** de 32 bits, aos quais atribuímos nomes como **PC, SP e MDR**.
  - Embora alguns desses nomes sejam familiares, é importante entender que esses registradores são acessíveis apenas no nível de microarquitetura (pelo microprograma).
  - Eles recebem esses nomes porque contêm um valor correspondente à variável do mesmo nome na arquitetura do nível de ISA.

O caminho de dados



- A maior parte dos registradores pode dirigir seu conteúdo para o **barramento B.**
- A saída da ULA comanda o **deslocador** e em seguida o **barramento C**, cujo valor pode ser escrito em um ou mais registradores ao mesmo tempo.

O caminho de dados



- A ULA precisa de duas entradas de dados: uma entrada esquerda (A) e uma entrada direita (B).
- Ligado à entrada esquerda está um registrador de retenção, H.
- Ligado à entrada direita está o barramento B, que pode ser carregado por cada uma de nove fontes, indicadas pelas nove setas cinza que chegam até ele.

O caminho de dados

- H pode ser carregado com a escolha de uma função da ULA que passe diretamente da entrada direita (vinda do barramento B) para a saída da ULA.
  - Uma função desse tipo: somar as entradas da ULA.

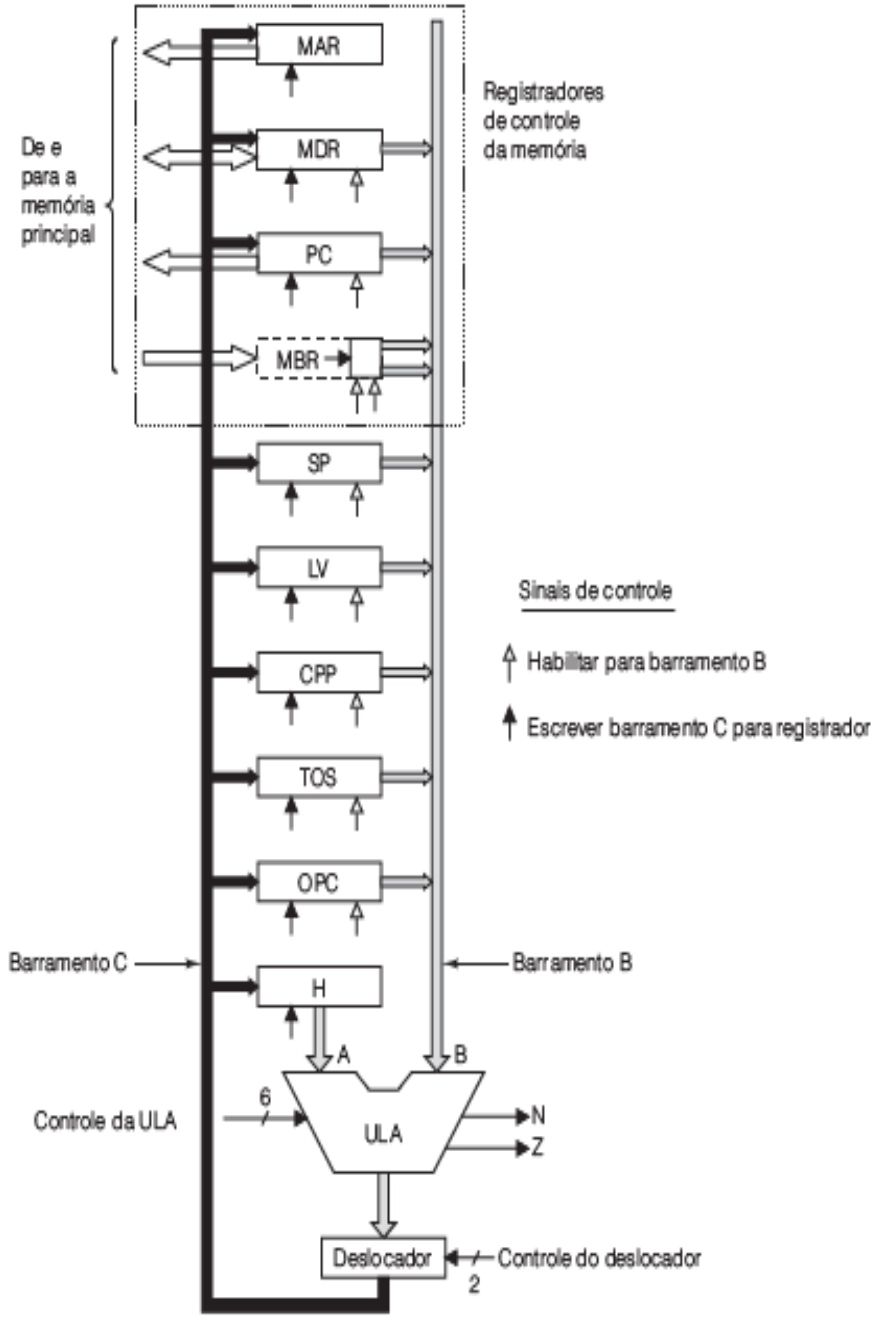
- É possível ler e escrever o mesmo registrador em um único ciclo?

Temporização

- É possível ler e escrever o mesmo registrador em um único ciclo?
  - Sim, por meio da temporização.
- Por exemplo, é permitido colocar SP no barramento B, habilitar o sinal INC (força um **vai-um** para o bit de ordem baixa, somando 1 ao resultado) e armazenar o resultado em SP, desse modo incrementando SP em 1.

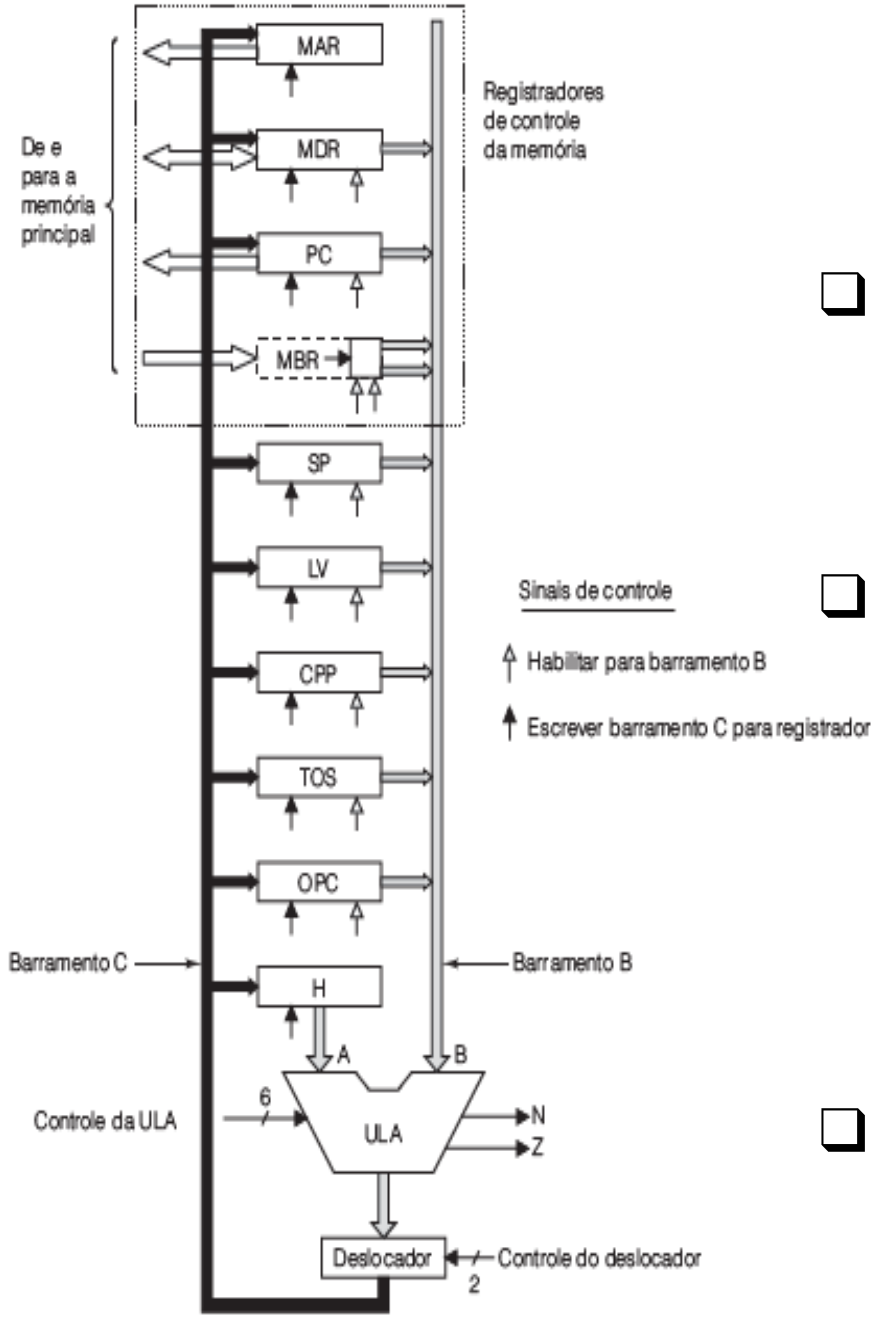
Temporização

- Como um registrador pode ser lido e escrito no mesmo ciclo sem produzir lixo?



Temporização

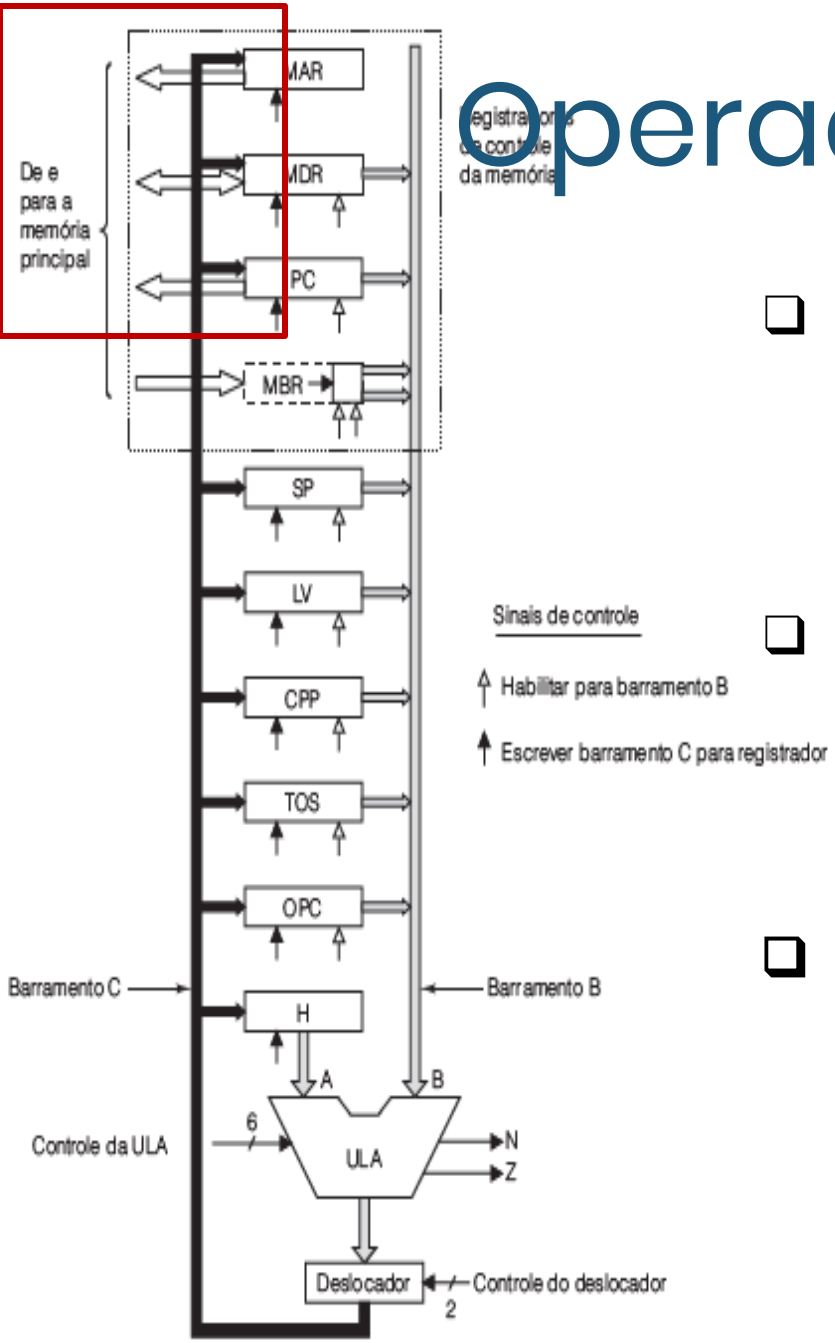
- Como um registrador pode ser lido e escrito no mesmo ciclo sem produzir lixo?
  - A solução é que leitura e escrita sejam executadas em instantes diferentes dentro do ciclo.
- Quando um registrador é selecionado como a entrada direita da ULA, seu valor é colocado no barramento B no início do ciclo e ali é mantido durante todo o ciclo.



Temporização

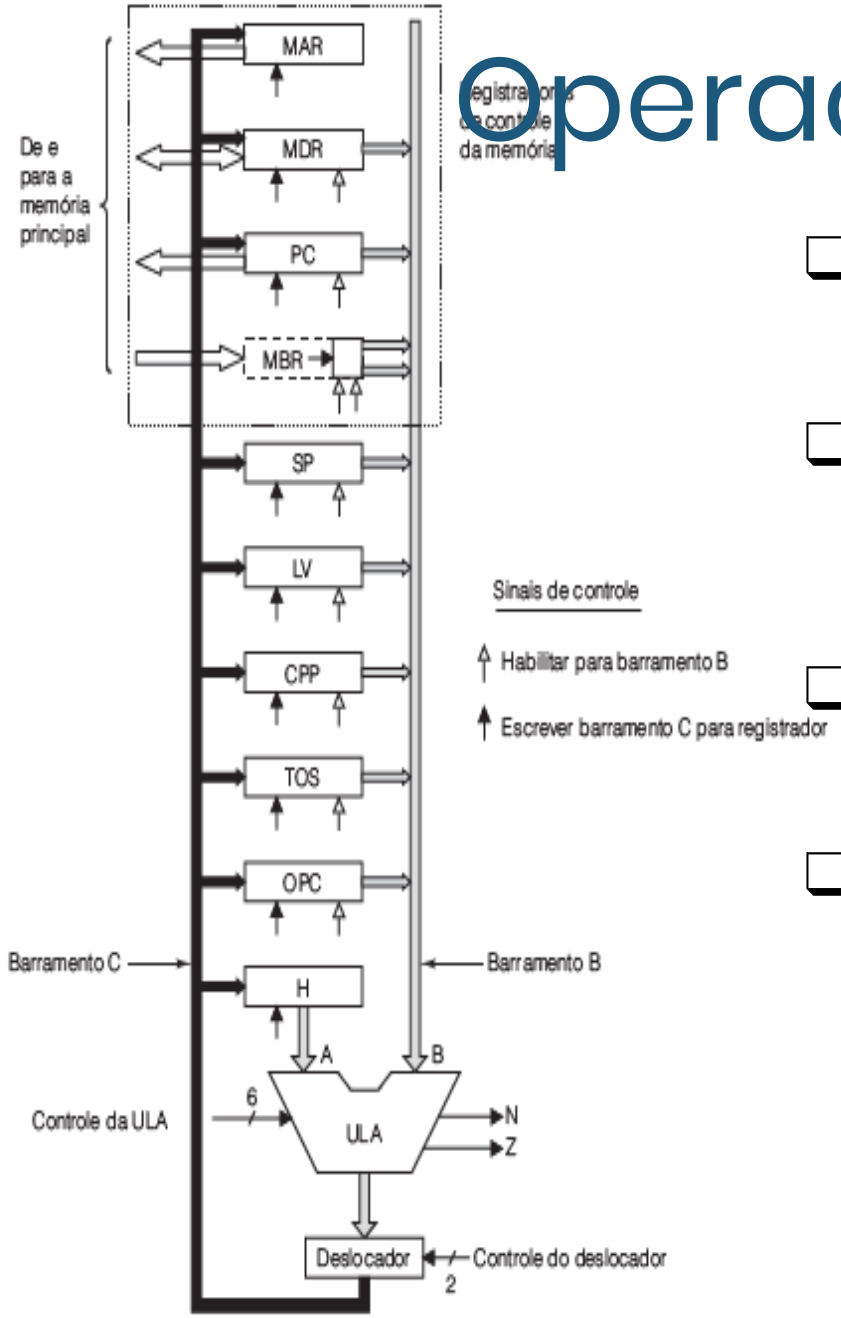
- Depois, a ULA realiza seu trabalho, produzindo um resultado que passa pelo deslocador e entra no barramento C.
- Próximo ao final do ciclo, quando se sabe que as saídas da ULA e deslocador são estáveis, um sinal de *clock* ativa o armazenamento do conteúdo do barramento C e o passa para um ou mais dos registradores.
- Um deles vai ser aquele que forneceu sua saída ao barramento B.

Operação de memória



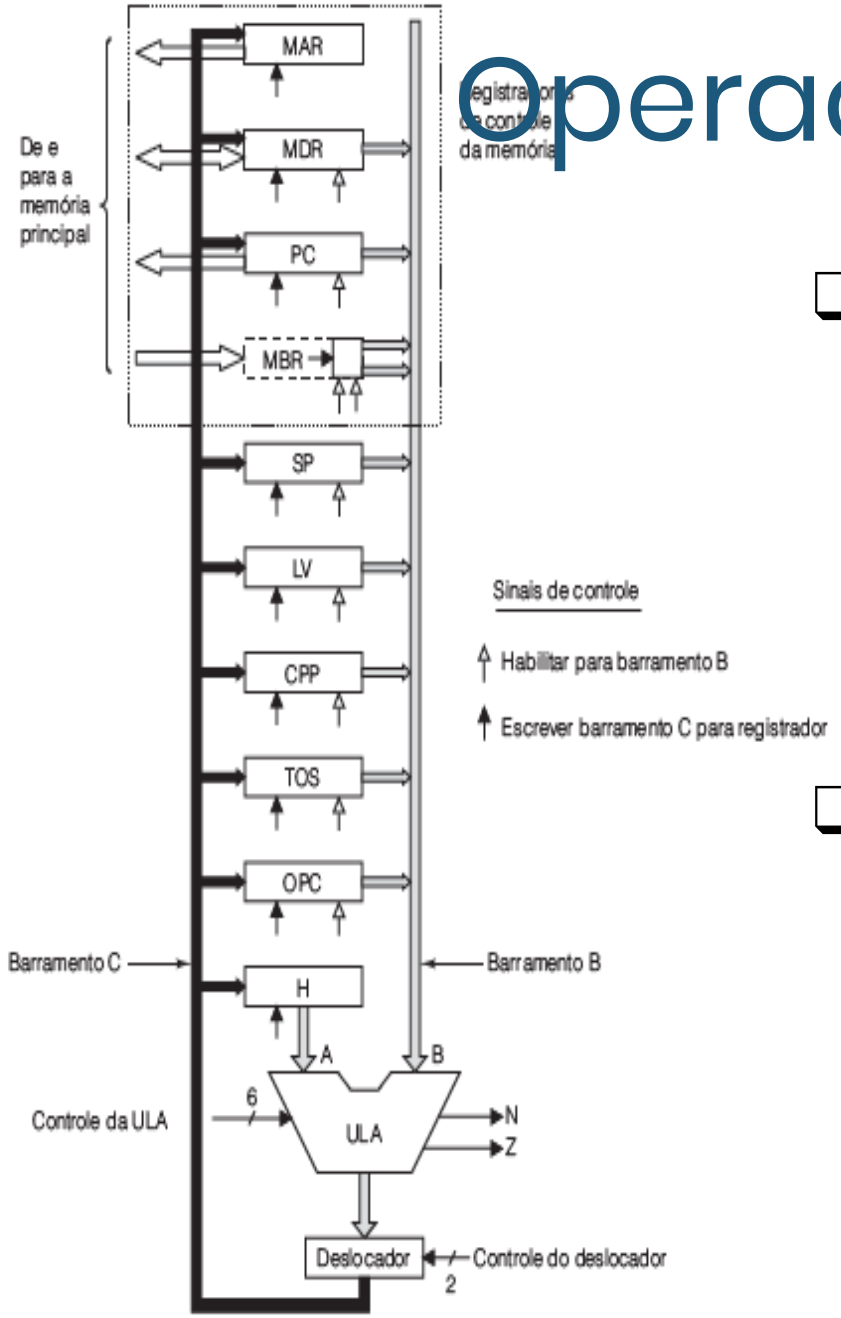
- Nossa máquina tem dois modos diferentes de se comunicar com a memória:
  - uma porta de memória de 32 bits, endereçável por palavra,
  - e outra de 8 bits, endereçável por byte.
- **A porta de 32 bits** é controlada por dois registradores, **MAR** (Memory Address Register – registrador de endereço de memória) e **MDR** (Memory Data Register – registrador de dados de memória)
- MBR é a **porta de 8 bits**, ela é controlada por um registrador **PC**, que lê 1 byte para os 8 bits de ordem baixa do MBR. Essa porta só pode ler dados da memória; ela não pode escrever dados na memória.

Operação de memória



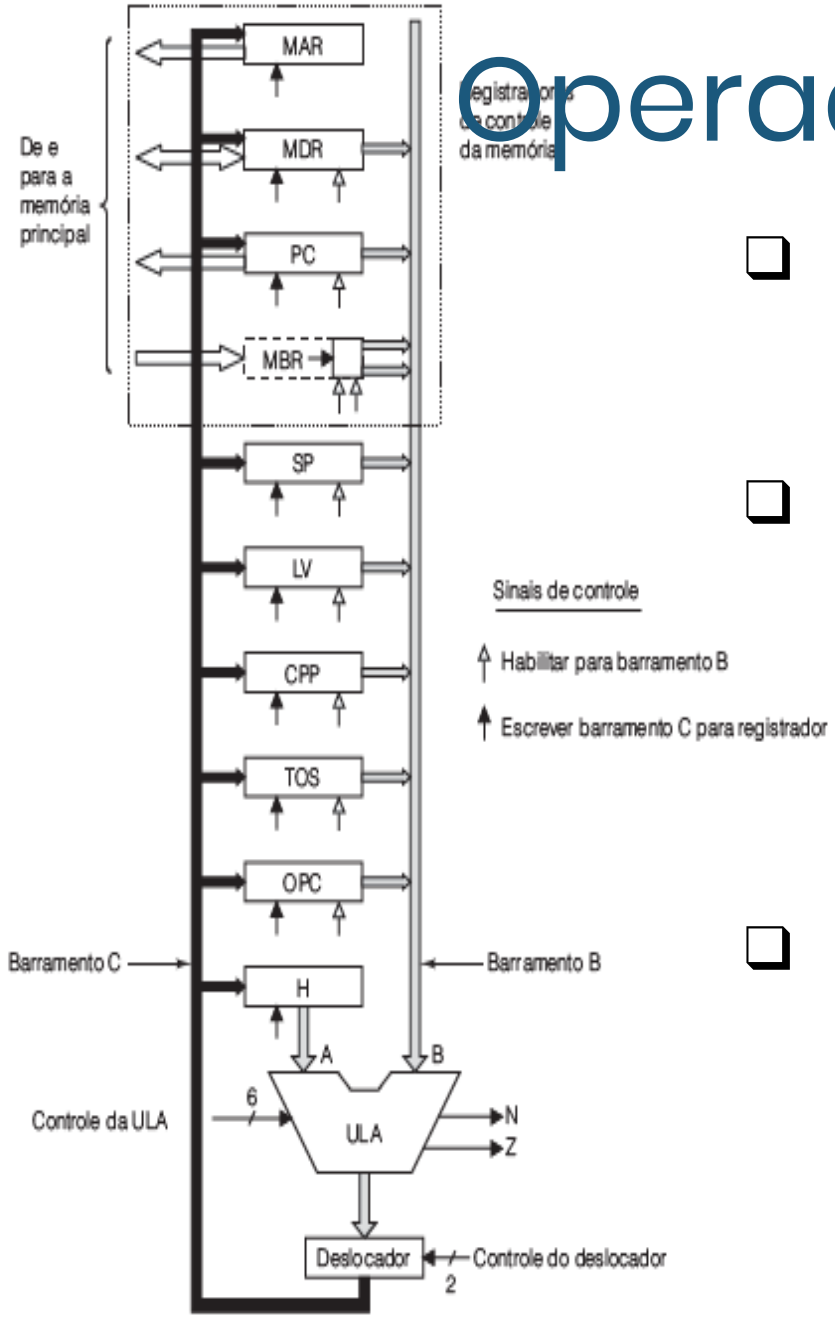
- Cada um desses registradores (e todos os outros) é comandado por um ou dois **sinais de controle.**
- Uma seta clara sob um registrador indica um sinal de controle que habilita a saída do registrador para o barramento B.
- Visto que MAR não tem conexão com o barramento B, não tem sinal de habilitação.
- H também não tem esse sinal porque está sempre habilitado, por ser a única entrada esquerda possível da ULA.

Operação de memória



- Uma seta negra sob um registrador indica um sinal de controle que escreve (carrega) no registrador a partir do barramento C.
  - MBR não pode ser carregado a partir do barramento C, pois não tem um sinal de escrita.
- Para iniciar uma leitura ou escrita da memória, os registradores de memória adequados devem ser carregados e em seguida deve ser emitido um sinal de leitura ou escrita para a memória.

Operação de memória



- MAR contém endereços de palavras, de modo que os valores 0, 1, 2 etc. se referem a palavras consecutivas.
- PC contém endereços de bytes, portanto, os valores 0, 1, 2 etc. se referem a bytes consecutivos.
  - Por exemplo, colocar um 2 em PC e iniciar uma leitura de memória, será executado -> leitura do byte 2 da memória e o colocará nos 8 bits de ordem baixa do MBR.
- Colocar 2 em MAR e iniciar uma leitura de memória lerá os bytes 8 –11 (a palavra aqui é de 4 bytes) da memória e os colocará em MDR.

Operação de memória

- Essa diferença de funcionalidade é necessária porque MAR e PC serão usados para referenciar duas partes diferentes da memória.
- A combinação MAR/MDR é usada para ler e escrever palavras de dados de nível ISA
- A combinação PC/MBR é empregada para ler o programa executável de nível ISA, que consiste em uma sequência de bytes.
- Todos os outros registradores que contêm endereços usam endereço de palavras, como o MAR.